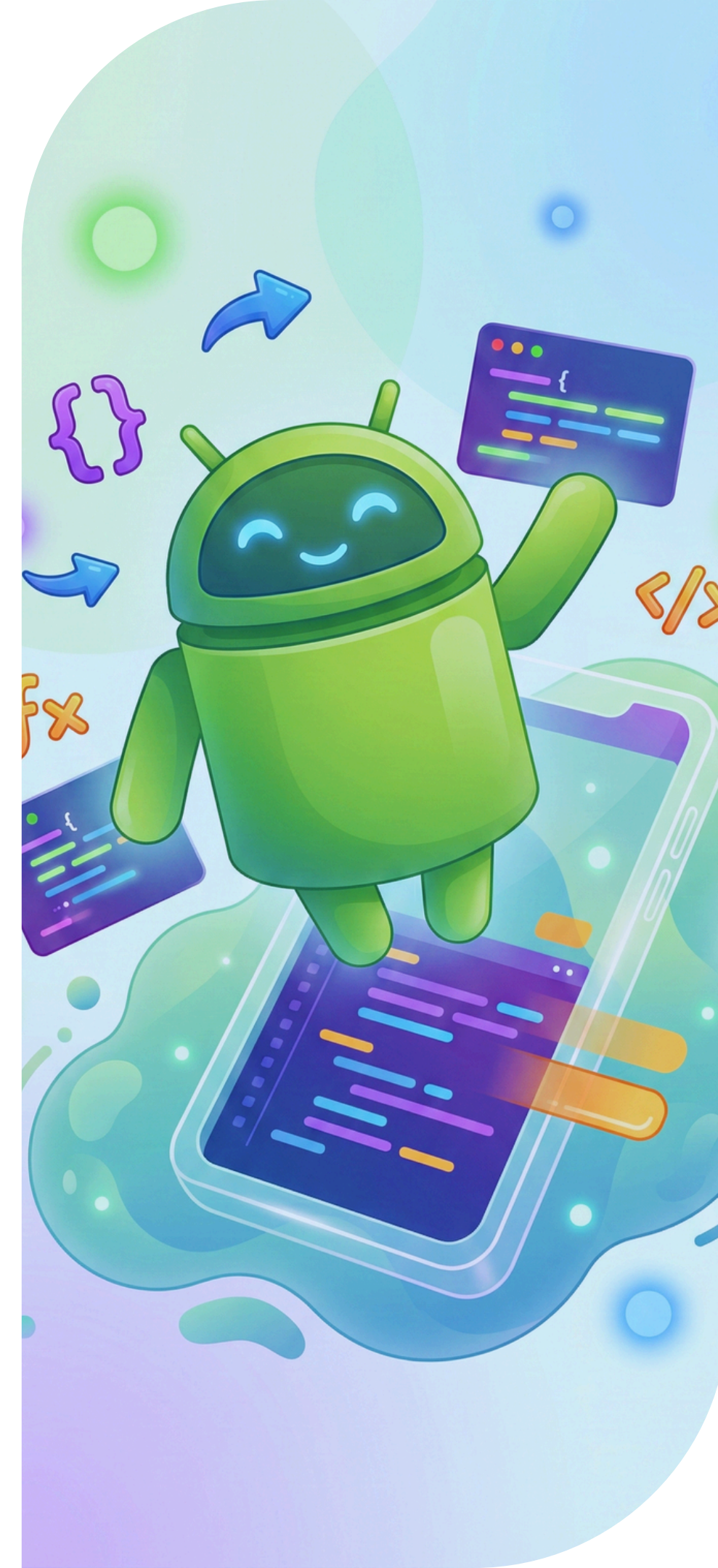


fit@hcmus

PHÁT TRIỂN PHẦN MỀM CHO THIẾT BỊ DI ĐỘNG

SEMINAR: JAVA/KOTLIN THREAD

Nhóm 6



NỘI DUNG

01

Giới thiệu & đặt vấn đề

02

Tạo và quản lý threads trong Kotlin

03

Dữ liệu chia sẻ và đồng bộ hóa

04

Giao tiếp giữa các threads

05

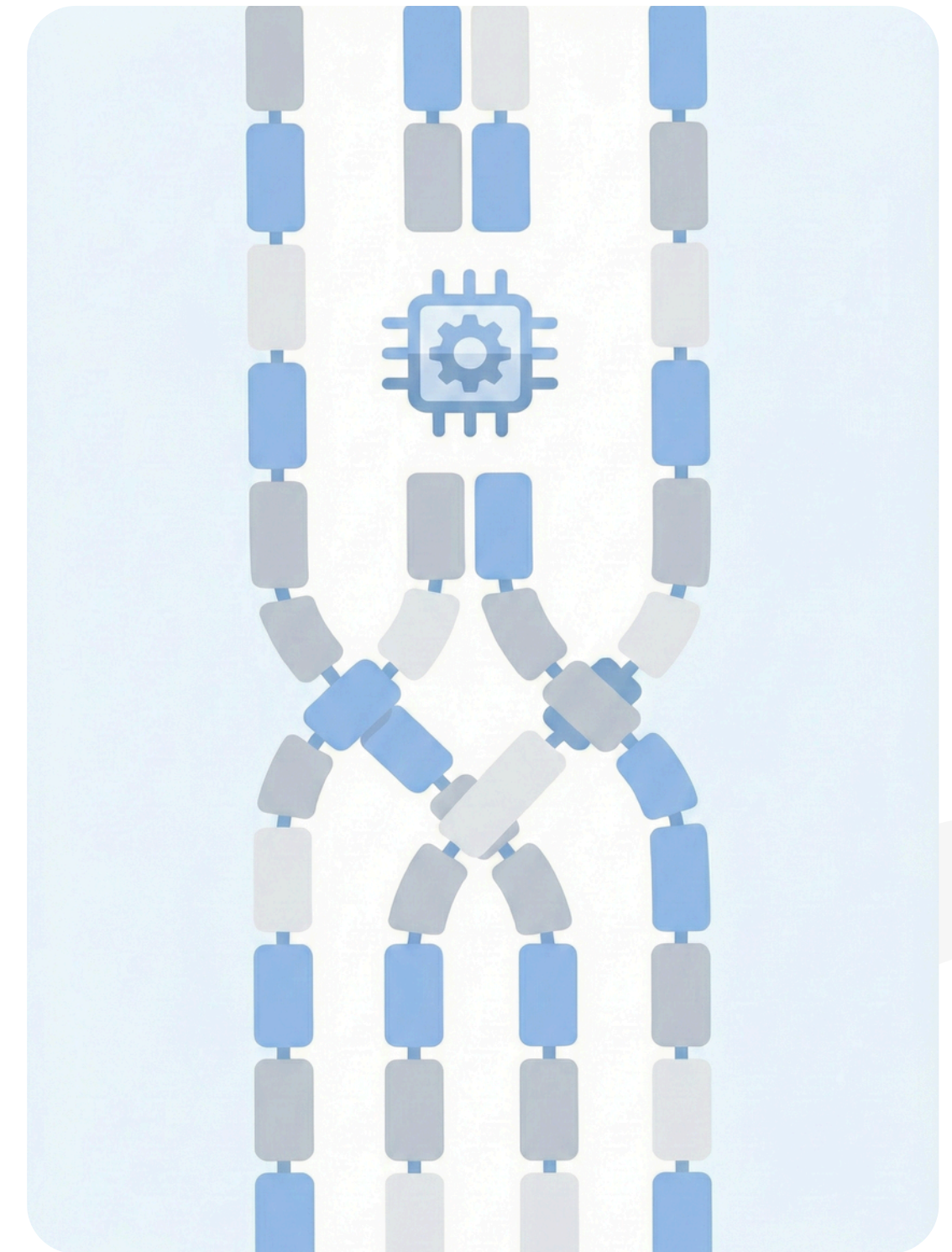
Các khái niệm nâng cao

06

Kết luận & định hướng tương lai

1 Đa luồng (Concurrency) là gì?

- **Định nghĩa:** Khả năng của chương trình xử lý nhiều tác vụ cùng một lúc.
- **Phân biệt:**
 - **Concurrency** (Đa luồng): Xử lý nhiều việc trong cùng một khoảng thời gian (có thể luân phiên).
 - **Parallelism** (Song song): Thực hiện nhiều việc chính xác cùng một lúc (yêu cầu phần cứng đa nhân).
- **Tại sao quan trọng?** Giúp ứng dụng phản hồi nhanh hơn và tận dụng tối đa tài nguyên CPU.



2 Tại sao chúng ta cần thread?

- Vấn đề của đơn luồng (Single-thread):
 - Nếu một tác vụ nặng (như tải file, xử lý ảnh) chạy trên luồng chính (Main Thread), giao diện người dùng (UI) sẽ bị đóng băng.
 - Nếu bạn chặn Main Thread quá 16ms -> Lag (Jank).
 - Nếu chặn quá 5 giây -> ANR (Application Not Responding).
- Giải pháp với Thread:
 - Đẩy tác vụ nặng xuống luồng nền (Background Thread).
 - Giữ luồng chính rảnh rỗi để phản hồi người dùng.
- Trong môi trường JVM: Mỗi Java/Kotlin Thread thường tương ứng trực tiếp với một Thread của hệ điều hành (OS Thread).

1 Cách tạo thread (Kiểu Java truyền thống)

- Sử dụng class **Thread** và interface **Runnable**.
- Đây là cách cơ bản nhất nhưng mã nguồn hơi dài dòng.

```
package com.example.seminar_threads_code

fun main() {
    // Create a Runnable using a lambda expression
    val myRunnable = Runnable {
        println("Running on Thread: ${Thread.currentThread().name}")
        Thread.sleep(1000) // Simulate some work with sleep
        println("Work completed!")
    }

    val thread = Thread(myRunnable)
    thread.start() // Start the thread
}
```

Ảnh 1: Tạo thread sử dụng Runnable

2 Cách tạo thread (Kiểu Kotlin - Helper function)

- Kotlin cung cấp hàm tiện ích `thread()` trong gói `kotlin.concurrent`.
- Ngắn gọn hơn, tự động gọi `start()` mặc định.

```
package com.example.seminar_threads_code

import kotlin.concurrent.thread

fun main() {
    // Create and start a thread using Kotlin's thread function
    thread(start = true, name = "MyKotlinThread") {
        println("Running on Thread: ${Thread.currentThread().name}")
        Thread.sleep(1000)
        println("Work completed!")
    }
}
```

Ảnh 2: Tạo thread sử dụng hàm tiện ích `thread()`

3 Vòng đời và điều khiển cơ bản

- Các trạng thái: New -> Runnable -> Running -> Blocked/Waiting -> Terminated.
- **Thread.sleep(millis)**: Tạm dừng thread hiện tại một khoảng thời gian.
- **thread.join()**: Bắt thread hiện tại (ví dụ: main) phải chờ cho đến khi thread kia chạy xong mới được đi tiếp.

```
package com.example.seminar_threads_code

import kotlin.concurrent.thread

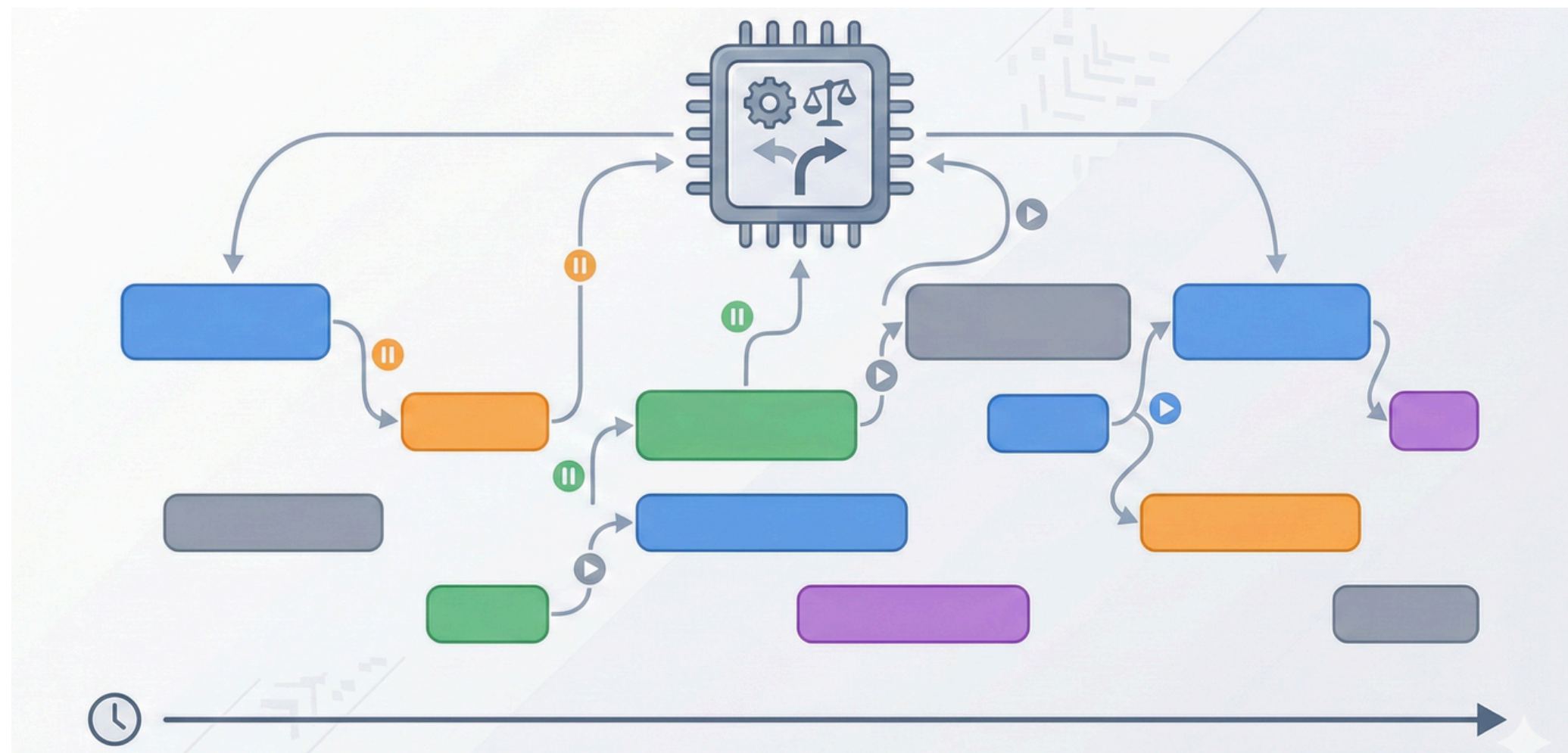
fun main() {
    val t = thread {
        Thread.sleep(2000)
        println("Child thread completed.")
    }

    println("Main thread is waiting for child thread to finish...")
    t.join() // Wait for the child thread to finish
    println("Main thread resumes after child thread completion.")
}
```

Ảnh 3: Sử dụng sleep() và join() trong thread

4 Demo thực tế - Cuộc đua giữa các Threads

Mục tiêu: Chứng minh rằng thứ tự thực thi của các thread là không thể đoán trước (Non-deterministic) và phụ thuộc vào Hệ điều hành (OS Scheduler).



Ảnh 4: Minh họa thứ tự thực thi các thread

4 Demo thực tế - Cuộc đua giữa các Threads

- **Sự xen kẽ (Interleaving):** Các dòng log không in ra theo thứ tự Xanh hết -> Đỏ hết -> Vàng hết. Chúng in lộn xộn vào nhau. Điều này chứng tỏ CPU đang chuyển đổi qua lại giữa các thread rất nhanh (Context Switching).
- **Thứ tự không đảm bảo:** Không ai biết trước thread nào sẽ chạy dòng code nào trước. Thread-Vàng được khởi tạo cuối cùng nhưng có thể chạy xong đầu tiên.
- **Main Thread không chờ:** Dòng log của Main thread ("KẾT THÚC CUỘC ĐUA") in ra xong xuôi trong khi các thread con vẫn đang chạy. Điều này nhắc lại khái niệm Bất đồng bộ (Asynchronous).

1 Vấn đề Race conditions

Xảy ra khi nhiều thread cùng truy cập và sửa đổi một biến chung (shared mutable state) cùng lúc. Điều này dẫn đến dữ liệu bị sai lệch không đoán trước được.

```
package com.example.seminar_threads_code

import kotlin.concurrent.thread

fun main() {
    var counter = 0
    val threads = mutableListOf<Thread>()

    // Create 10,000 threads, each incrementing the counter by 1
    // Expected final value of counter is 10,000
    for (i in 1..10000) {
        val t = thread {
            counter++ // This is not atomic and may lead to race conditions
        }

        threads += t
    }

    threads.forEach { it.join() } // Wait for all threads to finish
    println("Final value of counter: $counter")
}
```

Ảnh 5: Vấn đề khi nhiều thread sử dụng cùng một biến

2 Giải pháp đồng bộ hóa (Synchronized)

- Từ khóa **@Synchronized** (cho hàm) hoặc khối **synchronized(lock)**.
- Đảm bảo tại một thời điểm, chỉ có 1 thread được phép chạy đoạn code đó.

```
package com.example.seminar_threads_code

import kotlin.concurrent.thread

fun main() {
    var counter = 0
    val lock = Object()
    val threads = mutableListOf<Thread>()

    for (i in 1..10000) {
        val t = thread {
            synchronized(lock) {
                counter++ // Safe: Only one thread can increment at a time
            }
        }
        threads += t
    }

    threads.forEach { it.join() } // Wait for all threads to finish
    println("Final value of counter: $counter")
}
```

Ảnh 6: Đồng bộ hóa với synchronized() khi sử dụng thread

3 Các công cụ khác (Volatile & Atomic)

- **@Volatile**: Đảm bảo mọi thread đều nhìn thấy giá trị mới nhất của biến ngay khi nó thay đổi (nhưng không đảm bảo tính nguyên tử cho phép cộng/trừ).
- **Atomic Classes** (AtomicInteger, AtomicBoolean): Các lớp bọc sẵn an toàn cho thread, hiệu năng cao hơn synchronized cho các phép toán đơn giản.

```
package com.example.seminar_threads_code

import java.util.concurrent.atomic.AtomicInteger

fun main() {
    val atomicCounter = AtomicInteger(0)
    var regularCounter = 0

    // Create 100 threads, each incrementing both counters 1000 times
    val threads = List(100) {
        Thread {
            repeat(1000) {
                atomicCounter.incrementAndGet()
                regularCounter++ // race condition expected
            }
        }
    }

    // Start and join all threads
    threads.forEach { it.start() }
    threads.forEach { it.join() }

    // Atomic counter should be 100000 while regularCounter may be less
    println("Expected: ${100 * 1000}")
    println("Atomic: ${atomicCounter.get()}")
    println("Regular: $regularCounter")
}
```

Ảnh 7: Sử dụng Atomic classes

1 Cơ chế `wait()` và `notify()`:

- Dùng để phối hợp giữa các thread (một thread chờ tín hiệu từ thread khác).
- `wait()`: Nhả khóa và đi ngủ chờ tín hiệu.
- `notify()` / `notifyAll()`: Đánh thức các thread đang chờ dậy làm việc.

Lưu ý: Phải được gọi bên trong khối `synchronized`.

Giới thiệu &
đặt vấn đề

Tạo & quản lý
Threads trong
Kotlin

Dữ liệu chia sẻ và
đồng bộ hóa

Giao tiếp giữa
các threads

Các khái niệm
nâng cao

Kết luận và định
hướng tương lai

2 Ví dụ mô hình sản xuất - tiêu thụ (Producer - Consumer)

- Bao gồm 2 thread:
 - **Thread producer**: Sản xuất sản phẩm (thêm dữ liệu vào hàng đợi).
 - **Thread Consumer**: Tiêu thụ sản phẩm (lấy dữ liệu từ hàng đợi để xử lý).

```

package com.example.seminar_threads_code

import java.util.LinkedList
import kotlin.concurrent.thread

fun main() {
    val buffer = LinkedList<Int>()
    val capacity = 5
    val lock = Object()

    // Producer Thread
    thread {
        while (true) {
            synchronized(lock) {
                while (buffer.size == capacity) lock.wait() // Wait if full
                val item = (1..100).random()
                buffer.add(item)
                println("Added item $item. Current buffer size: ${buffer.size}")
                lock.notifyAll() // Notify consumer threads
            }
            Thread.sleep(500) // Simulate time taken to produce an item
        }
    }

    // Consumer Thread
    thread {
        while (true) {
            synchronized(lock) {
                while (buffer.isEmpty()) lock.wait() // Wait if empty
                val item = buffer.removeFirst()
                println("Removed item $item. Current buffer size: ${buffer.size}")
                lock.notifyAll() // Notify producer threads
            }
            Thread.sleep(1000) // Simulate time taken to consume an item
        }
    }
}

```

Ảnh 8: Cài đặt mô hình sản xuất - tiêu thụ

Giới thiệu &
đặt vấn đề

Tạo & quản lý
Threads trong
Kotlin

Dữ liệu chia sẻ và
đồng bộ hóa

Giao tiếp giữa
các threads

Các khái niệm
nâng cao

Kết luận và định
hướng tương lai

1 Tại sao cần thread pool?

- Việc tạo mới một thread (`new Thread()`) tốn rất nhiều tài nguyên hệ thống (RAM, CPU overhead).
- **Thread Pool**: Tạo sẵn một nhóm các thread và tái sử dụng chúng cho nhiều tác vụ khác nhau.

2 Sử dụng Executors

- **`Executors.newFixedThreadPool(n):`**
Tạo pool với cố định n thread.
- **`Executors.newCachedThreadPool():`**
Tạo pool co giãn theo nhu cầu.

```
package com.example.seminar_threads_code

import java.util.concurrent.Executors

fun main() {
    // Create a Thread Pool with 2 threads
    val executor = Executors.newFixedThreadPool(2)

    // Submit 10 tasks to the pool
    for (i in 1..10) {
        executor.submit {
            println("Task $i is running on Thread: ${Thread.currentThread().name}")
            Thread.sleep(500)
        }
    }

    executor.shutdown() // Shutdown the executor after tasks are done
}
```

Ảnh 9: Tạo thread pool với Executors

Giới thiệu &
đặt vấn đề

Tạo & quản lý
Threads trong
Kotlin

Dữ liệu chia sẻ và
đồng bộ hóa

Giao tiếp giữa
các threads

Các khái niệm
nâng cao

Kết luận và định
hướng tương lai

1 Tổng kết ưu và nhược điểm

- **Ưu điểm:**

- Mô hình quen thuộc, có sẵn trong Java/Kotlin standard library.
- Kiểm soát chi tiết việc thực thi.

- **Nhược điểm:**

- **Tốn kém:** Mỗi thread tốn khoảng 1-2MB RAM stack size. Không thể tạo hàng ngàn thread (C10k problem).
- **Phức tạp:** Dễ gặp lỗi Deadlock, Race Conditions.
- **Blocking:** Khi chờ I/O, thread bị chặn hoàn toàn, gây lãng phí tài nguyên.

Giới thiệu &
đặt vấn đề

Tạo & quản lý
Threads trong
Kotlin

Dữ liệu chia sẻ và
đồng bộ hóa

Giao tiếp giữa
các threads

Các khái niệm
nâng cao

Kết luận và định
hướng tương lai

2 Về bài thuyết trình tiếp theo

- Các vấn đề trên dẫn đến sự ra đời của một giải pháp hiện đại hơn.

→ **Kotlin Coroutines:**

- "Lightweight threads" (Luồng siêu nhẹ).
- Viết code bất đồng bộ theo phong cách tuần tự.
- Không chặn luồng (Non-blocking).

THANK YOU

FOR YOUR ATTENTION

